```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt
import psutil
import optuna
from pysr import PySRRegressor
import shap
import warnings

warnings.filterwarnings("ignore")  # Suppress any future warnings for cleaner output

# --------------------------------
# Helper: System Resource Monitor
def check_system_resources():
    memory = psutil.virtual_memory()
    cpu = psutil.cpu_percent(interval=1)
    print(f"CPU Usage: {cpu}%")
    print(f"Memory Usage: {memory.percent}%")
    return cpu, memory.percent

# --------------------------------
# Step 1: Load the GZ dataset
gz_df = pd.read_csv("GZ_gzdv1-2_cleaned.csv", low_memory=False)
print("Columns in GZ DataFrame:", gz_df.columns.tolist())
print(gz_df.head())

# --------------------------------
# Step 2: Select and preprocess relevant features
selected_features = ['z', 'rMag', 'Ar', 'Pth100', 'Smooth', 'Featured']
gz_df = gz_df[selected_features].copy()

# Impute missing values
gz_df[selected_features] =
SimpleImputer(strategy='mean').fit_transform(gz_df[selected_features])

# --------------------------------
# Step 3: Feature Engineering
gz_df['morph_sum'] = gz_df[['Smooth', 'Featured']].sum(axis=1)
gz_df['dust_corrected_mag'] = gz_df['rMag'] - gz_df['Ar']
```

```
# Normalize both variables
scaler_norm = MinMaxScaler()
gz_df[['morph_sum_norm', 'Pth100_norm']] = scaler_norm.fit_transform(gz_df[['morph_sum',
'Pth100']])

# Rank idea 1: average of normalized metrics
gz_df['cosmo_rank_avg'] = (gz_df['morph_sum_norm'] + gz_df['Pth100_norm']) / 2

# Rank idea 2: weighted, give more influence to structure or environment
gz_df['cosmo_rank'] = 0.7 * gz_df['Pth100_norm'] + 0.3 * gz_df['morph_sum_norm']

# Ensure valid magnitude values before applying expected SFR formula
gz_df = gz_df[gz_df['dust_corrected_mag'].notna() & (gz_df['dust_corrected_mag'] < 100)]

# Compute expected SFR and apply log transform to avoid overflow
gz_df['expected_SFR'] = 1e-4 * (10 ** (0.4 * (22 - gz_df['dust_corrected_mag']))) ** 1.4
gz_df['expected_SFR'].replace([np.inf, -np.inf], np.nan, inplace=True)
gz_df.dropna(subset=['expected_SFR'], inplace=True)

# Apply log transformation to expected_SFR to avoid numerical instability
gz_df['expected_SFR'] = np.log1p(gz_df['expected_SFR'])  # log1p for safety

# Define features and target
X = gz_df[['z', 'rMag', 'Ar', 'Pth100', 'morph_sum']]
y = gz_df['expected_SFR']

# ------------------------------
# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ------------------------------
# Step 5: Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ------------------------------
# Step 6: Random Forest - Optuna
check_system_resources()

best_rf = None
try:
    def objective_rf(trial):
```

```
        model = RandomForestRegressor(
            max_depth=trial.suggest_int("max_depth", 5, 30),
            n_estimators=trial.suggest_int("n_estimators", 50, 500),
            min_samples_split=trial.suggest_int("min_samples_split", 2, 20),
            random_state=42,
            n_jobs=2
        )
        return np.mean(cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2',
n_jobs=2))

    study_rf = optuna.create_study(direction="maximize")
    study_rf.optimize(objective_rf, n_trials=50)

    best_rf = RandomForestRegressor(**study_rf.best_params, random_state=42, n_jobs=2)
    best_rf.fit(X_train_scaled, y_train)
except Exception as e:
    print(f"Random Forest training failed: {e}")

# --------------------------------
# Step 7: Gradient Boosting - Optuna
best_gb = None
try:
    def objective_gb(trial):
        model = GradientBoostingRegressor(
            max_depth=trial.suggest_int("max_depth", 3, 20),
            n_estimators=trial.suggest_int("n_estimators", 50, 500),
            min_samples_split=trial.suggest_int("min_samples_split", 2, 20),
            learning_rate=trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
            random_state=42
        )
        return np.mean(cross_val_score(model, X_train_scaled, y_train, cv=3, scoring='r2',
n_jobs=2))

    study_gb = optuna.create_study(direction="maximize")
    study_gb.optimize(objective_gb, n_trials=50)

    best_gb = GradientBoostingRegressor(**study_gb.best_params, random_state=42)
    best_gb.fit(X_train_scaled, y_train)
except Exception as e:
    print(f"Gradient Boosting training failed: {e}")

# --------------------------------
# Step 8: Evaluate Models
if best_rf is not None:
```

```
    y_pred_rf = best_rf.predict(X_test_scaled)
    print(f"Random Forest - MSE: {mean_squared_error(y_test, y_pred_rf):.4f}, R²:
{r2_score(y_test, y_pred_rf):.4f}")

if best_gb is not None:
    y_pred_gb = best_gb.predict(X_test_scaled)
    print(f"Gradient Boosting - MSE: {mean_squared_error(y_test, y_pred_gb):.4f}, R²:
{r2_score(y_test, y_pred_gb):.4f}")

# ------------------------------
# Step 9: SHAP Analysis
try:
    if best_rf is not None:
        explainer_rf = shap.Explainer(best_rf, X_train_scaled)
        shap_values_rf = explainer_rf(X_test_scaled)
        shap.plots.beeswarm(shap_values_rf, show=False)
        plt.title("SHAP Summary - Random Forest")
        plt.savefig("shap_summary_rf.png")
        plt.close()
        pd.DataFrame(shap_values_rf.values, columns=X.columns).to_csv("shap_values_rf.csv",
index=False)

    if best_gb is not None:
        explainer_gb = shap.Explainer(best_gb, X_train_scaled)
        shap_values_gb = explainer_gb(X_test_scaled)
        shap.plots.beeswarm(shap_values_gb, show=False)
        plt.title("SHAP Summary - Gradient Boosting")
        plt.savefig("shap_summary_gb.png")
        plt.close()
        pd.DataFrame(shap_values_gb.values,
columns=X.columns).to_csv("shap_values_gb.csv", index=False)
except Exception as e:
    print(f"SHAP analysis failed: {e}")

# ------------------------------
# Step 10: Symbolic Regression with PySR
try:
    symbolic_model = PySRRegressor(
        model_selection="best",
        niterations=500,
        population_size=2000,
        binary_operators=["+", "-", "*", "/"],
        unary_operators=["sqrt", "log", "exp", "sin", "cos"],
        loss="loss(x, y) = (x - y)^2",
```

```
        maxsize=30,
        verbosity=1,
        procs=4,
        random_state=42
    )

    symbolic_model.fit(X_train_scaled, y_train)
    print("Best symbolic models:\n", symbolic_model)

    y_pysr_pred = symbolic_model.predict(X_test_scaled)
    print(f"PySR R² score: {r2_score(y_test, y_pysr_pred):.4f}")

    symbolic_model.save("pysr_model.pkl")
    symbolic_model.equations_.to_csv("pysr_equations.csv", index=False)
    print("Top PySR Equations:\n", symbolic_model.equations_.head())
except Exception as e:
    print(f"Symbolic regression failed: {e}")

# ------------------------------
# Step 11: Visual Comparison of R² Scores
try:
    model_names = []
    r2_scores = []

    if best_rf is not None:
        model_names.append("Random Forest")
        r2_scores.append(r2_score(y_test, y_pred_rf))
    if best_gb is not None:
        model_names.append("Gradient Boosting")
        r2_scores.append(r2_score(y_test, y_pred_gb))
    if 'y_pysr_pred' in locals():
        model_names.append("PySR")
        r2_scores.append(r2_score(y_test, y_pysr_pred))

    if model_names:
        plt.bar(model_names, r2_scores, color=['skyblue', 'salmon',
'limegreen'][:len(model_names)])
        plt.ylabel("R² Score")
        plt.title("Model Performance Comparison")
        plt.ylim(0, 1)
        plt.savefig("model_comparison_r2.png")
        plt.close()
except Exception as e:
    print(f"Model comparison plot failed: {e}")
```

```
# ------------------------------
# Final system resource check
check_system_resources()
```